

Module 4: Arrays, Methods, Parameters, Return Values, Searching, Sorting and Matrix Operations

Set A

1. Student Marks Analysis

A class teacher wants to analyze the marks obtained by students in a programming course. Develop an application that accepts the marks of N students and provides the following features: display all student marks, find the highest and lowest marks, calculate the class average, count the number of students who scored above the average.

Requirements:

- Use a 1D array to store the marks of N students.
- Implement each operation as a separate method that accepts the array (and other needed values) as parameters and returns the required result.

2. Employee Attendance Record

A company records the attendance of employees for a month. Each employee's attendance is represented as: 1 → Present, 0 → Absent.

Develop a program to: display the attendance record, calculate the total number of days present, calculate the attendance percentage, and identify whether the employee satisfies the minimum attendance requirement of 75%.

Requirements:

- Use a 1D array of 0s and 1s.
- Implement separate methods for display, present-count, percentage calculation, and eligibility check — each using parameters and return values.

3. Hospital Patient Temperature Monitoring

A hospital records the temperature of patients admitted to different wards. Develop an application that: stores patient temperatures in an array, displays all temperatures, finds the highest and lowest temperatures, counts patients with fever (temperature $\geq 38^{\circ}\text{C}$), and displays the average temperature.

Requirements:

- Use a 1D array to store N patient temperatures.
- Implement each operation (display, highest/lowest, fever-count, average) as a separate method using parameters and return values.

Set B

1. Supermarket Weekly Sales Matrix

A supermarket records the daily sales (in ₹) of different product categories over a week. Each row represents a product category, and each column represents a day's sales.

Develop an application to: accept the sales data, display the sales matrix, calculate the total weekly sales for each product category, calculate the total sales for each day, identify the best-selling product category, and identify the day with the highest sales.

Requirements:

- Use a 2D array (rows = categories, columns = days).
- Implement each major operation using separate methods with appropriate parameters and return values.

2. Multiplex Seat Reservation

A multiplex represents its seating arrangement using a two-dimensional array. The application should: display the seating layout, reserve a seat specified by the user, prevent booking an already reserved seat, and display the updated seating arrangement.

Requirements:

- Use a 2D array to represent the seating layout (e.g., 0 = available, 1 = reserved).
- Implement seat booking, seat display, and availability-check as separate methods.

3. University Attendance Matrix

A university records the attendance of students over five working days. Each row in the matrix represents a student, and each column represents a day's attendance: 1 indicates present, 0 indicates absent.

Develop an application to: accept the attendance matrix, display it in tabular form, calculate the attendance percentage of each student, determine the day with the highest overall attendance, and identify students who meet the minimum attendance requirement of 75%.

Requirements:

- Use a 2D array (rows = students, columns = days).
- Implement separate methods for reading, displaying, computing per-student attendance percentage, and finding the best-attendance day — each using parameters and return values.

Set C

1. Bank Transaction Search and Sort

A bank stores the daily transaction amounts of a customer for a month. Develop an application that allows the user to: display all transactions, calculate total credits and total debits, find the highest and lowest transaction, search for a particular transaction amount, and display the transactions in ascending order.

Requirements:

- Use a 1D array to store the month's transactions.
- Use separate methods for searching (linear search) and sorting (any simple sort), each with clear parameters and return values.

2. Multi-Specialty Hospital Diagnostic Analysis

A multi-specialty hospital stores the diagnostic test results of patients in a matrix. Each row represents a patient, and each column represents a diagnostic test score.

Develop an application that allows the administrator to: accept the diagnostic score matrix, display it in tabular format, calculate the average score for each patient, calculate the average score for each diagnostic test, identify the patient with the highest overall average, identify the diagnostic test with the highest average score, and display all patients whose average score exceeds a user-specified threshold.

Requirements:

- Use modular programming — implement separate methods for: matrix input, matrix display, patient-wise analysis, test-wise analysis, average calculation, and report generation.

3. University Grading System

A university stores the marks of students in five subjects using a two-dimensional array. Develop an application that: accepts the marks of N students, calculates the total and average marks for each student, assigns grades based on the average, displays the topper(s), calculates the average marks for each subject, and identifies the subject with the highest class average.

Requirements:

- Use a 2D array (rows = students, columns = subjects).
- Implement all major operations using separate methods with appropriate parameters and return values.